

TU DRESDEN · MS COMPUTER SCIENCE

Digitization and Data Analytics

Distributed Computing II — Complete Study Guide

Hadoop · MapReduce · Spark · Flink · HPC/Big-Data Convergence

In-depth lecture notes, annotated diagrams, and exam-oriented Q&A
(conceptual, comparative, and research-level questions)

Compiled from: "Digitization and Data Analytics" — Lecture 11, Distributed Computing II, CIDS/ZIH

Table of Contents

1. Foundations: Data Pipeline & Scale-Out Paradigm

2. Hadoop & MapReduce

- 2.1 Origins: GFS & MapReduce papers
- 2.2 Hadoop 1.0 Ecosystem
- 2.3 HDFS Design Assumptions and Goals
- 2.4 MapReduce Word-Count Walkthrough
- 2.5 Limitations & Extensions of MapReduce

3. In-Memory Processing

4. Apache Spark

- 4.1 Architecture & Cluster Managers
- 4.2 Spark Context, Driver, Executors
- 4.3 RDDs — Resilient Distributed Datasets
- 4.4 Data Structure Evolution: RDD → DataFrame → Dataset
- 4.5 Spark Streaming

5. Apache Flink & Stream Processing

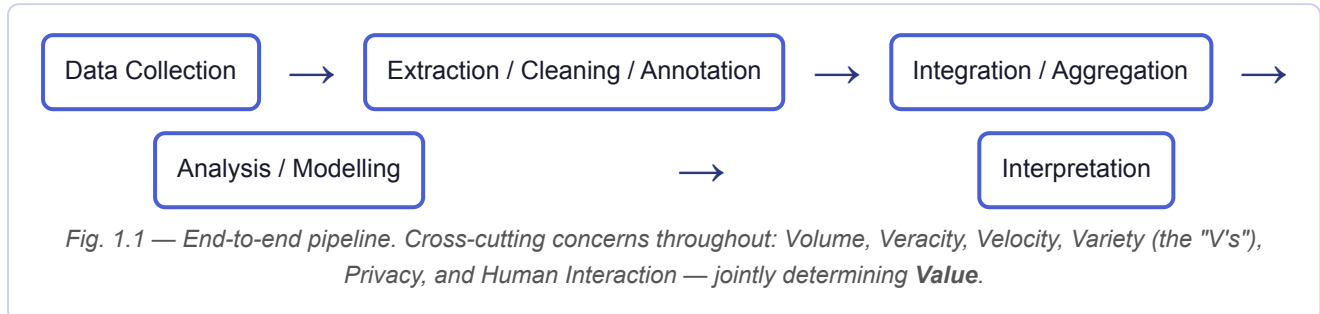
6. HPC vs. Big Data: Scheduling & Convergence

7. Master Comparison Cheat-Sheets

8. Exam & Research Q&A Bank

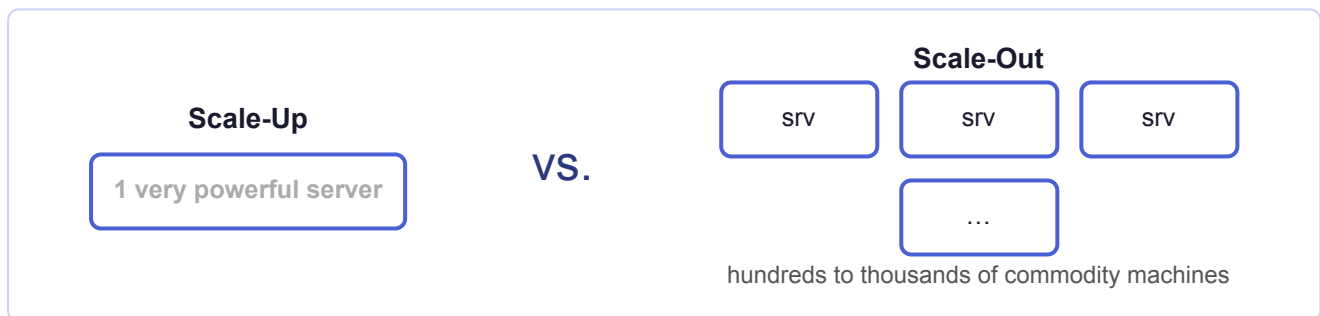
1. Foundations: Data Processing Pipeline & Scale-Out Paradigm

1.1 The Data Processing Pipeline (recap)



Every stage of the pipeline must cope with the "V" characteristics of Big Data. **Volume** is the sheer size of data; **Velocity** is the rate at which it arrives; **Variety** is heterogeneity of format/source; **Veracity** is trustworthiness/noise. These four properties, together with privacy constraints and the degree of human-in-the-loop interaction, ultimately determine whether the pipeline produces *value*.

1.2 Scale-Out vs. Scale-Up



Rather than relying on a single, expensive, highly reliable machine (scale-up), Big Data infrastructures follow the **scale-out paradigm**: many (hundreds to thousands) low-cost, commodity machines are used together. The system as a whole must be designed to **not rely on any single instance** — individual node failure is expected and must be tolerated transparently.

WHY NOT JUST BUY BETTER HARDWARE? Cost scales super-linearly with single-machine performance (high-end SMPs, exotic interconnects), while commodity components scale roughly linearly in cost per unit of aggregate throughput. Scale-out also gives near-linear increases in both storage and I/O bandwidth as nodes are added — something a single powerful machine cannot do beyond its physical bus/disk limits.

2. Hadoop & MapReduce

2.1 Origins: The Two Foundational Papers

Paper	Authors / Venue	Contribution
"The Google File System"	Ghemawat, Gobioff, Leung — SOSP 2003	Distributed, fault-tolerant file system for very large files across commodity clusters; foundation later cloned as HDFS
"MapReduce: Simplified Data Processing on Large Clusters"	Dean, Ghemawat — OSDI 2004	Programming model + runtime that hides parallelisation, fault-tolerance, data distribution and load balancing behind two simple functions: map and reduce

Apache Hadoop is the open-source framework that clones both of these Google systems (HDFS clones GFS; Hadoop MapReduce clones Google's MapReduce). It is designed to process very large amounts of unstructured/complex data on a large number of machines.

2.2 Hadoop 1.0 Ecosystem

Layer	Components
Coordination	ZooKeeper
Applications / higher-level frameworks	Workflow (Oozie), Graph (Giraph), SQL/DWH (Hive), Scripting (Pig), ...
Distributed data processing model & job scheduling	Hadoop MapReduce, HBase (column store)
Distributed storage	HDFS
Serialization	Avro
Physical layer	Shared-nothing cluster of commodity servers

Fig. 2.1 — Hadoop is not a single tool, but a layered, interoperable ecosystem: physical storage (HDFS) at the bottom, a processing/scheduling layer (MapReduce, HBase) above it, and specialised higher-level frameworks (Hive, Pig, Giraph, Oozie) on top.

KEY IDEA The move from "just a function" (map/reduce) to an **interoperable framework stack** is itself significant: it allows different programming paradigms (SQL-like, graph, scripting, workflow orchestration) to share the same underlying distributed storage and execution engine.

2.3 HDFS/Hadoop Design Assumptions and Goals

These assumptions are the "why" behind almost every Hadoop/HDFS design decision — a very common examination topic.

Assumption / Goal	Explanation
Hardware failure is the norm	Clusters have hundreds to thousands of commodity servers; at that scale, some component is essentially always failing. The system must detect faults and recover automatically rather than treating failure as exceptional.
Streaming data access	Applications need high-throughput streaming (sequential) access to data, not low-latency random access. HDFS is optimized for batch throughput, not interactive/transactional use.
Large data sets	Designed for tens of millions of files, each ranging from gigabytes to terabytes — i.e., aggregate cluster capacity in the petabyte range.
Moving computation to data	It's cheaper to move a small program to where the (large) data resides than to move large volumes of data to the program. This requires the framework to know data locality (which node/rack holds which block).
Portability	Runs across heterogeneous hardware and software/OS platforms, easing adoption.

TYPICAL EXAM QUESTION "Explain why HDFS favours throughput over latency, and how this differs from a classical relational database design." — Answer should connect the streaming-access assumption to batch analytics workloads versus OLTP.

2.4 MapReduce Word-Count Walkthrough

Input	Splitting	Mapping	Shuffling (sync!)	Reducing	Result
"Deer Bear River Car Car River Deer Car Bear"	3 line-splits (parallel)	(Deer,1) (Bear,1) (River,1) (Car,1)(Car,1) (River,1) (Deer,1)(Car,1) (Bear,1)	group by key across all mappers	Bear:2, Car:3, Deer:2, River:2 (parallel per key)	Combined final counts

Fig. 2.2 — Splitting and Mapping are "pure" parallel stages (no cross-node communication needed). Shuffling is the synchronization point: data must be redistributed across the network so all values for the same key land on the same reducer. Reducing is again parallel per key.

WHY SHUFFLE MATTERS The shuffle phase is the only stage requiring network-wide data movement/synchronization — it is usually the main performance bottleneck of classic MapReduce jobs (disk I/O + network + sort).

2.5 Limitations and Extensions to MapReduce

Geoffrey Fox (Indiana University) classifies MapReduce-style computation into six patterns, showing how far the model has been stretched beyond the original batch use case:

#	Pattern	Description	Typical use
1	Map Only	Purely embarrassingly-parallel map, no reduce/sync	Image processing, format conversion
2	Classic MapReduce	Map → shuffle → reduce, one pass	Word count, log analysis
3	Iterative MapReduce / Map-Collective	Repeated map-reduce cycles feeding back into next iteration	Machine learning (k-means, PageRank), Spark's core strength
4	Point-to-Point / Map-Communication	Local, graph-structured communication instead of global shuffle	Graph algorithms (Giraph/Pregel-style)
5	Map-Streaming	Maps continuously communicate through message brokers on unbounded event streams	Storm, Kafka Streams, Flink
6	Shared-memory / Map-Communicate	Workers share state via in-memory structures rather than re-reading from disk	In-memory processing — direct motivation for Spark

RESEARCH ANGLE Classic MapReduce (pattern 2) re-reads/re-writes intermediate results to disk between every stage — disastrous for *iterative* algorithms (ML, graph) that re-use the same data across many rounds. Patterns 3 and 6 (iteration + shared/in-memory state) are exactly the gap that Apache Spark was built to fill.

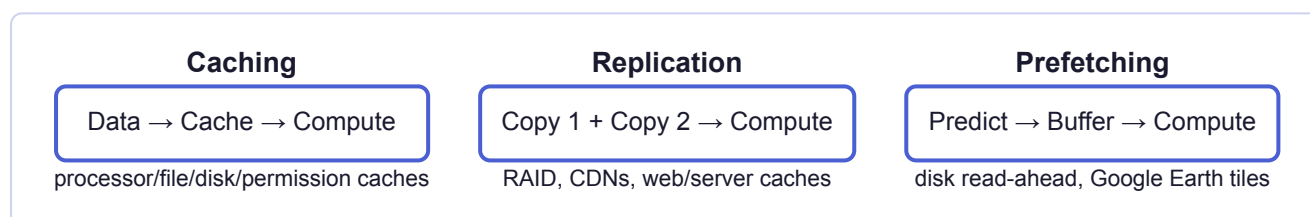
3. In-Memory Processing

3.1 From Disk-Based to In-Memory Analytics

	Previously (disk-based)	Now (in-memory)
Storage	Relational DB on disk	Data held resident in RAM
Query paradigm	SQL, but slows down as data grows	Near real-time analysis, no IO-wait bottleneck
Enabler	—	RAM cost has fallen enough to be viable at scale
Scope	Separate compute stages	Basis for full pipelines: iterations, joins, etc.

Historical driver: Business Intelligence (BI) needs. Classic BI reports were submitted and results came back later — the famous *"coffee-break analytics"* pattern (start a report, go get a coffee, come back to the result). In-memory processing was pursued specifically to eliminate this latency.

3.2 In-Memory (Hardware-Level) Techniques



Technique	Limitation
Caching	Only works if the "working set" is small (repeated access to the same copies)
Prefetching	Only helps when access patterns are predictable
Replication	Expensive, and bounded by the capacity of receiving-side machines
General	Efficiency always depends on the actual application/workflow behaviour — no one-size-fits-all trick

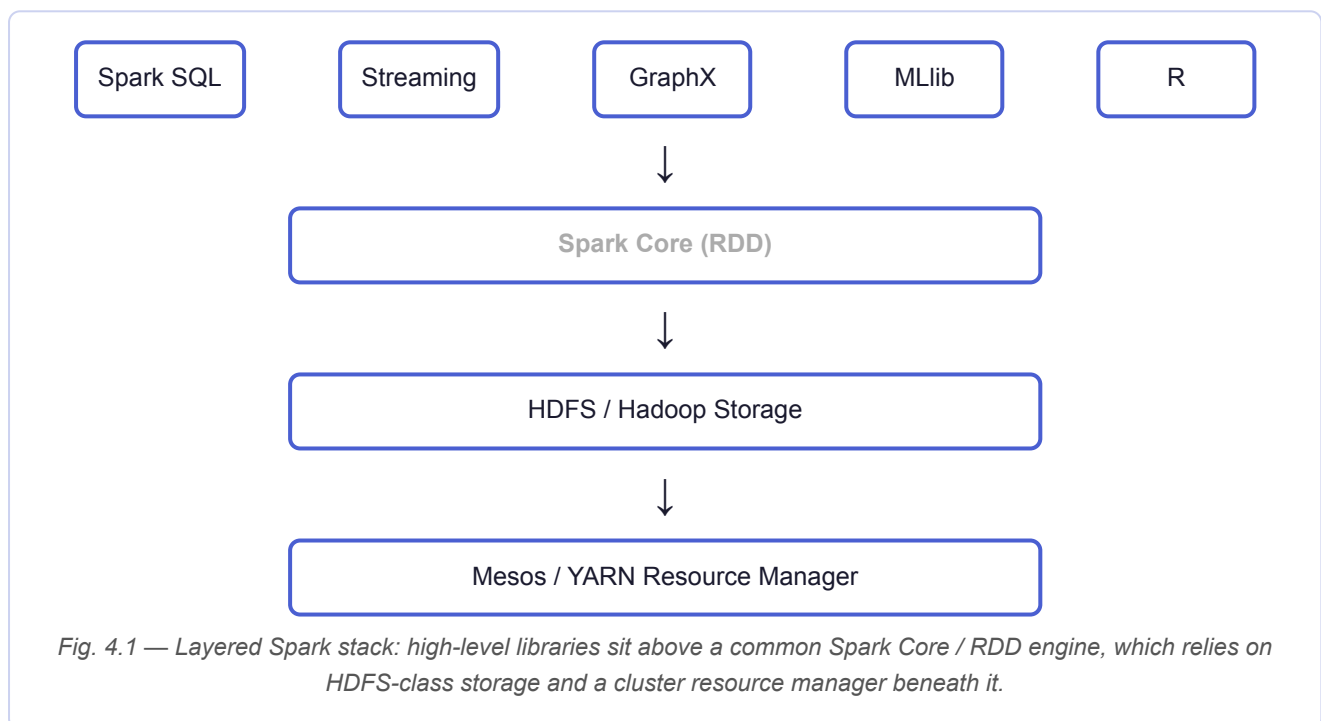
EXAM ANGLE Be ready to argue *why* in-memory processing frameworks (Spark) outperform disk-based MapReduce specifically for **iterative** workloads: each MapReduce iteration re-reads/re-writes to HDFS, while an in-memory framework keeps the working set (e.g., an RDD) cached in RAM across iterations.

4. Apache Spark

4.1 Overview & Architecture

Apache Spark is the **third generation** of scale-out data processing frameworks (after "simple function" MapReduce, and the ecosystem-of-tools Hadoop 1.0 stage): a **fast, general-purpose engine** for large-scale data processing.

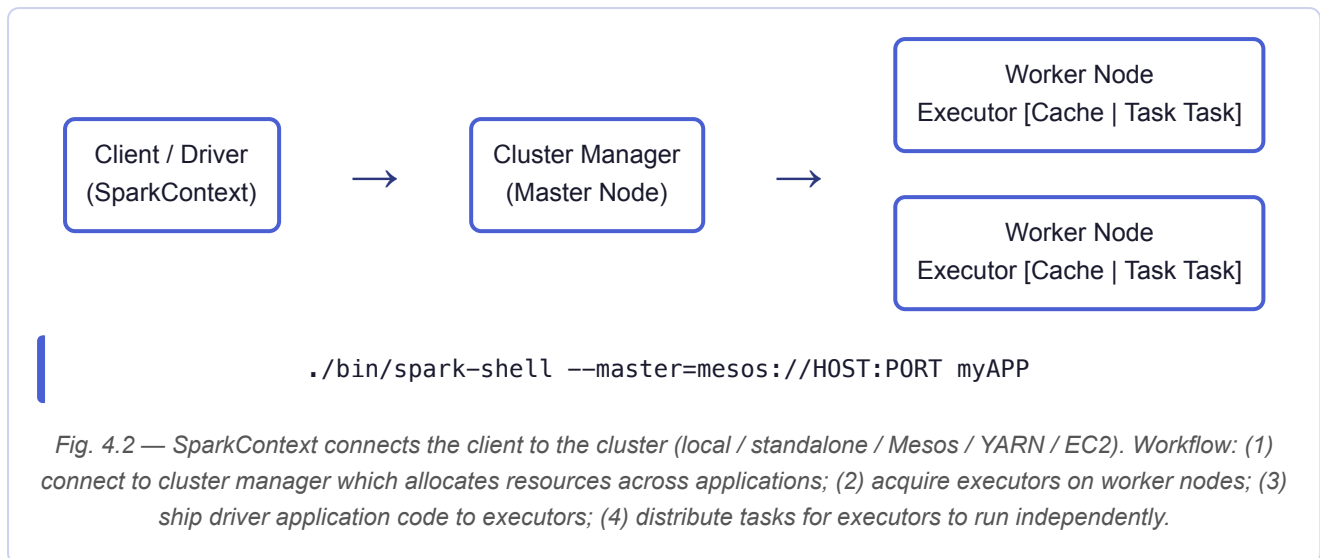
- Created by AMPLab, UC Berkeley (2009); commercialized as **Databricks**
- Written in **Scala**, licensed under the Apache Foundation, large international open-source community
- Offers a unified stack: **Spark SQL, Streaming, GraphX, MLlib, (Spark)R** — all on top of **Spark Core**
- Data access via HDFS, Cassandra, HBase, Hive, Tachyon, or any Hadoop-compatible data source
- Runs standalone, or on a cluster manager: **Hadoop YARN, Apache Mesos**, or on EC2



Spark provides a **Directed Acyclic Graph (DAG) execution engine** supporting cyclic data flow and in-memory computing, plus over 80 high-level operators — usable interactively from Scala, Python, or R shells. Its libraries combine seamlessly within a single application (e.g., run SQL, then feed the result directly into MLlib).

BENCHMARK RESULT (FROM SPARK HOMEPAGE) For an iterative workload, Hadoop MapReduce took ~110s vs. Spark's ~0.9s for a comparable operation — because MapReduce re-reads/writes disk on every iteration while Spark keeps data cached in memory. As iterations increase (1→30), the runtime gap between Hadoop and Spark widens dramatically (near-linear growth for Hadoop vs. much flatter growth for Spark).

4.2 Spark-Context: Driver, Cluster Manager, Executors



Application isolation

Each Spark **application has its own private driver and its own executors** — even when co-located on the same physical servers. This provides isolation between concurrently running applications sharing cluster storage, but also means Spark applications cannot trivially share cached data with each other (unlike, say, a shared external cache).

4.3 RDD — Resilient Distributed Dataset

RDD is Spark's core abstraction and gives the framework its name-worthy properties:

Property	Meaning
Resilient	Fault-tolerant via the RDD lineage graph : Spark remembers the sequence of transformations used to build an RDD, so a lost partition can be <i>recomputed</i> from its lineage rather than restored from a replica.
Distributed	Data resides across multiple nodes in the cluster, split into partitions.
Dataset	A collection of partitioned records — primitives, tuples, or arbitrary objects.

Parallelism is controlled by the RDD (e.g. via **repartition**). Spark tries to schedule computation on already-partitioned data to avoid unnecessary data exchange (data locality optimisation, echoing the Hadoop "move computation to data" principle).

Transformations vs. Actions

Type	Behaviour	Examples
Transformations	Lazy — return another RDD, nothing computed yet	<code>map</code> , <code>filter</code> , <code>flatMap</code>
Actions	Trigger computation and return a concrete value	<code>collect</code> , <code>reduce</code> , <code>saveAsTextFile</code>

```

lines = spark.textFile("hdfs://...") // RDD
errors = lines.filter(_.startsWith("ERROR")) // Transformation
errors.filter(_.contains("HDFS")).map(_.split('\t')(3)).collect() // Transformation

```

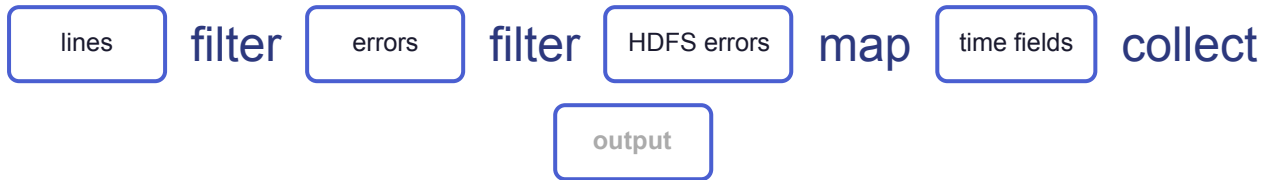


Fig. 4.3 — Each transformation builds a new (immutable) RDD holding a pointer back to its parent — this chain is the lineage graph. Only the final action (`collect()`) actually triggers execution of the whole chain.

CLASSIC EXAM QUESTION "Why are Spark transformations lazy?" — Laziness lets Spark build the full DAG of operations *before* executing anything, enabling global optimisation (e.g., pipelining narrow transformations, avoiding unnecessary intermediate materialization) and letting the scheduler decide the most data-local/efficient execution plan.

4.4 Data Structure Evolution: RDD → DataFrame → Dataset

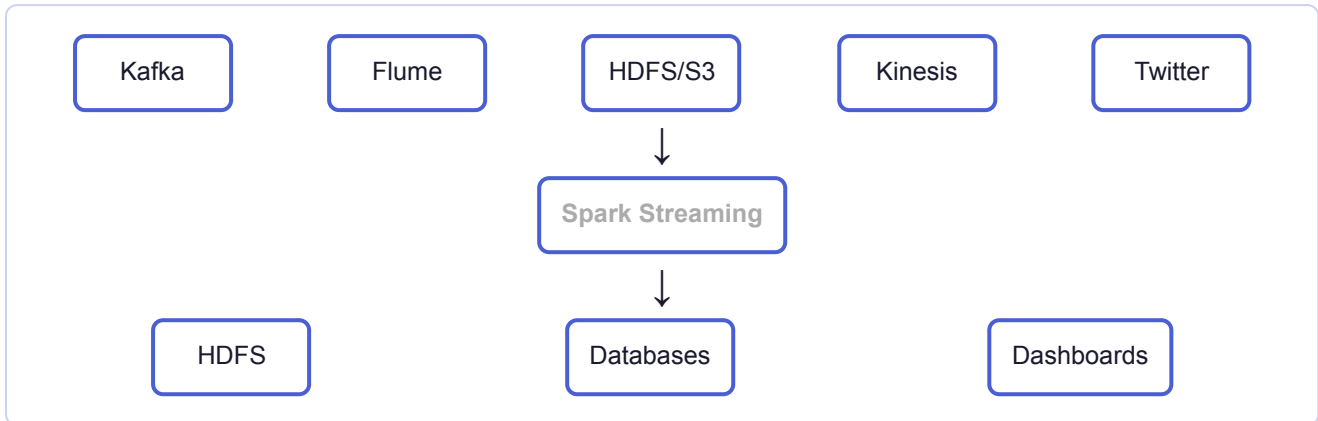
Structure	Year	Characteristics
RDD	2011	Distributed collection of JVM objects; functional operators (map, filter, ...); fully type-safe but no schema/optimizer awareness
DataFrame	2013	Distributed collection of Row objects; expression-based operations + UDFs; benefits from Spark's logical-plan optimizer (Catalyst); fast/efficient internal (columnar-ish) representation, but not compile-time type-safe
Dataset	2015	Internally rows, externally typed JVM objects — "best of both worlds": type-safe + fast (combines RDD's compile-time safety with DataFrame's optimizer benefits)

Since Spark 1.4, this enhanced stack (SQL / Python / R / Streaming / Advanced Analytics → DataFrame & Dataset → **Tungsten Execution Engine**) introduced:

- **Memory management & binary processing:** leveraging known application semantics (schema) to manage memory explicitly, eliminating JVM object overhead and garbage-collection pressure.
- **Cache-aware computation:** algorithms/data structures exploit the CPU memory hierarchy (L1/L2/L3 caches).
- **Code generation:** generating specialized bytecode at runtime to exploit modern compilers/CPU's instead of interpreting generic expression trees.

RESEARCH ANGLE The RDD→DataFrame→Dataset evolution mirrors a broader trend in big-data engines: trading a small amount of flexibility (arbitrary JVM objects) for large gains in optimizability (a known, structured schema lets the engine apply relational-database-style query optimization and vectorized/compiled execution).

4.5 Spark Streaming



Spark Streaming extends Spark Core to streaming sources. Crucially, it does **not** process events one at a time; instead **input is split into "mini-batches"** (DStreams), each processed as a small Spark job by the same underlying Spark Engine. It additionally introduces **window semantics**: a windowed operation is computed periodically over a sliding window across several of these underlying micro-batches (e.g., window of 3 time units, sliding every 1 unit).

KEY DISTINCTION TO REMEMBER Spark Streaming = **micro-batching**, not "true" per-event streaming. This is the crucial architectural contrast with Flink (Section 5).

5. Apache Flink & Stream Processing

5.1 Batch vs. Stream Processing Model

	Batch Processing	Stream Processing
Data	"Fixed" (bounded, stored)	Streaming through the system (unbounded)
Query	Workload of queries run into the system on demand	Continuous / standing query, always active
Resources	Full dataset available	Limited resources for state (can't hold everything)
Model	Query → DB	Events → Queries (↔ State) → Events

5.2 Stream Processing Alternatives

System	Character
Apache Storm	True streaming, low latency, lower throughput. Low-level API (Bolts, Spouts) + Trident for higher-level abstractions.
Spark Streaming	Stream processing layered on top of a batch engine → high throughput but higher latency. Functional API (DStreams), restricted by the underlying batch runtime.
Apache Samza	True streaming, built on Apache Kafka; state is a first-class citizen. Slightly different stream notion; low-level API.
Apache Flink	True streaming with an <i>adjustable</i> latency–throughput trade-off. Rich functional API exploiting the streaming runtime (e.g., rich windowing semantics).

5.3 Apache Flink

- Large-scale data processing engine, similar in ambition to Spark
- Derived from the earlier academic **Stratosphere** project
- Supports iterations (machine learning), graph processing, and batch processing — not just streaming
- **Streams results between operators** — i.e., data can flow operator-to-operator without waiting for a whole stage/batch to complete

Layer	Examples
Applications & Languages	Hive, Cascading, Giraph, Mahout, Pig, Crunch
Data processing engines	MapReduce, Flink, Spark, Storm, Tez
App / resource management	YARN, Mesos
Storage / streams	HDFS, HBase, Kafka

Fig. 5.1 — Flink sits at the same "data processing engine" layer as Spark/Storm/MapReduce, reusing the same resource-management (YARN/Mesos) and storage (HDFS/HBase/Kafka) layers.

Flink's headline properties: **true streaming** with adjustable latency/throughput, a rich functional API, **flexible windowing semantics**, **exactly-once processing guarantees** using (deliberately small) checkpointed state, and native data-stream outputs.

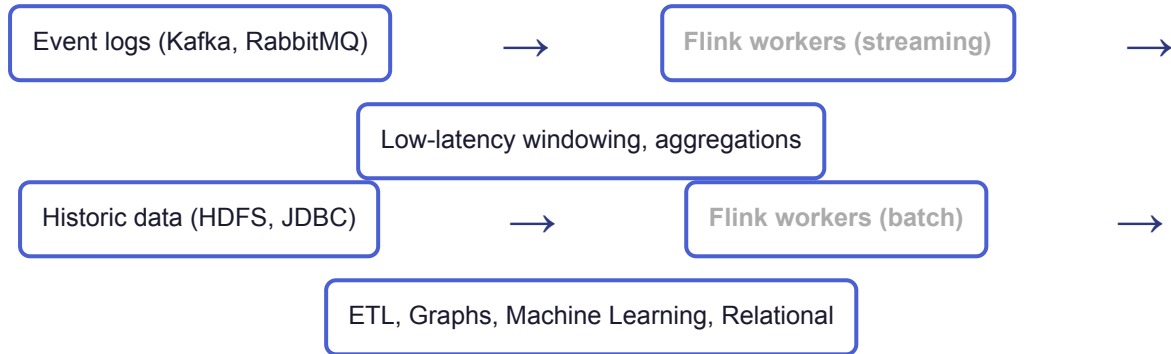


Fig. 5.2 — Flink treats batch as a special (bounded) case of streaming, unifying both under one runtime (Marton Balassi, data Artisans).

Worked example: stock price stream

Two raw sources — a socket feed of "HDP, 23.8" strings and a live source of typed `StockPrice(SPX, 2113.9)` events — are (1) parsed, then (2)/(3) merged from multiple stock sources into (4) one unified `StockStream`.

That unified stream is then windowed (a 10-second window recomputed every 5 seconds) and split three ways: (2) global `MinByPrice`, (3) `MaxByPrice` grouped by symbol, (4) `MeanPrice` grouped by symbol — demonstrating Flink's flexible, per-key windowed aggregation semantics.

5.4 Spark vs. Flink Streaming — Architectural Contrast

	Apache Flink	Spark Streaming
Model	Native/true streaming — direct API to access events as they arrive	Micro-batching — streaming mapped onto a sequence of short batch processes
Latency	Lower, tunable	Higher (bounded by micro-batch interval)
Throughput	High	High

Native Streaming (Flink): events processed continuously as they arrive (one flow, per-event processing).

Micro-batching (Spark): events are grouped into `Batch 1, Batch 2, Batch 3, ...`; a discrete "Process" step triggers after each batch boundary.

Fig. 5.3 — (after Gorasiya 2019) *Native streaming vs. micro-batching: the fundamental architectural split behind most Flink-vs-Spark comparisons.*

Benchmark: Karimov et al. (ICDE 2018) — "Benchmarking Distributed Stream Data Processing Systems"

- Compared Storm, Spark, and Flink implementations of a **windowed aggregation** query (10-minute tumbling window, grouped SUM) at increasing cluster sizes (2/4/8 nodes) and at both **maximum throughput** and **90% of maximum throughput**.
- **Result:** both Spark and Flink scale well with additional nodes (latency variance decreases as cluster size grows). Spark showed a **slight latency offset** compared to Flink under equivalent load — consistent with its micro-batching architecture adding a small, structural scheduling delay per batch.

RESEARCH ANGLE This benchmark is a good citation for any answer arguing that "true streaming" engines are not simply *faster* in absolute throughput than micro-batch engines, but rather offer a more favourable **latency profile** at comparable throughput and scaling behaviour — the real trade-off is architectural (per-event vs. per-batch), not raw performance.

APPLICATION DRIVER True low-latency stream processing (pattern (5) Map-Streaming from Section 2.5) is essential for use cases such as **autonomous driving** and **real-time sensor data analysis**, where micro-batch latencies of hundreds of milliseconds to seconds are unacceptable.

5.5 Big Data Streaming — Use Case: Traffic Monitoring

Motivated by the growing availability of sensors and cyber-physical systems (toll gantries, buoys, weather stations, smart meters, robots, wearables, radio telescopes, ...). A concrete example:

- **Applications:** toll collection, speed control, traffic-jam detection, route planning
- **Data collection:** fixed toll stations, smartphones (crowd-sourced GPS traces), logistics/fleet tracking (GPS)

This is a canonical **Variety + Velocity** problem: heterogeneous sources (fixed sensors vs. mobile phones vs. logistics devices) producing continuous streams that must be fused and analysed close to real time.

6. HPC vs. Big Data: Scheduling & Convergence

6.1 Differences between HPC and Big Data Scheduling

	HPC	Big Data (e.g., Hadoop)
Resource control	Fine-grained (cores, accelerators, memory, wall-clock time, ...)	Coarser-grained, job/task oriented
Scheduler examples	Slurm, Grid Engine, Moab, LoadLeveler	Hadoop v1: integrated scheduler — master Job Tracker communicating with Task Trackers on each node
Fault handling	Typically checkpoint/restart at application level	Job Tracker natively manages/re-schedules failed tasks (fail-safe by design)
Languages	C/C++, Fortran; both SIMD and MIMD supported; increasingly Python	Java, Scala, Python
Parallel model	MPI = flexible MIMD (arbitrary communication)	MapReduce can be classified as an SIMD (single-instruction, multiple-data) style problem — the same map function applied uniformly across data

6.2 Convergence Patterns — Application/User Perspective

- Traditionally, data and compute were in close proximity (same data center); this is no longer guaranteed in cloud/edge settings.
- "Data concentration" is reduced towards the **edge** — huge volumes are generated far from the central HPC/cloud facility.
- Developments are driven by the Big Data aspects **Velocity**, **Veracity**, **Variety** (heterogeneous data).
- Dealing with the data "deluge" is *harder* at the edges of the data space — this is where **Edge Computing** becomes attractive: pre-process near the source before shipping data centrally.
- Edge devices may themselves produce very high data rates: e.g. microscopes, gene sequencers, sensor networks.

Distributed edge devices (weather stations, gene sequencers, smartphones, industrial sensors, telescopes, ...) $\rightleftharpoons$ HPC Center / Cloud, trading off **latency & cost** against where processing actually happens ("locus of processing": in-situ / in-transit / in-locus).

Fig. 6.1 — BDEC (Big Data and Extreme-scale Computing) view of Volume/Veracity/Velocity/Variety pushing processing decisions outward from the HPC center toward the network edge.

6.3 HPC + HDA (High-end Data Analytics) Workflows

- Analytics scenarios are **not singular applications** but complex, multi-step workflows spanning the full data life cycle.

- Need for a **"user in the loop"**: confirming model iterations or the outcome of predictive analytics before proceeding.
- Potentially **highly iterative** (reflecting Section 2.5's iterative MapReduce pattern).
- Must be flexible enough to react to changes in the software stack: **interoperability of tools**, **reproducibility of results**, **adaptability to systems**.

The classic empirical-science loop, annotated with HPC/HDA roles: **Observations** (from instruments/sensors/records) → **Abduction** (HDA: discovery of patterns/anomalies, making guesses) → **Theories** (generation of explanatory hypothesis, formulated as a mathematical model) → **Deduction** (HPC: executing the model to generate predictions) → **Predictions** → **Induction** (HDA: assimilating predictions and data to evaluate the hypothesis) → back to Observations/Reality.

Fig. 6.2 — HPC handles the deductive, compute-heavy simulation side; HDA (Big-Data analytics) handles the inductive/abductive, data-heavy discovery side. Real workflows continuously cycle between the two.

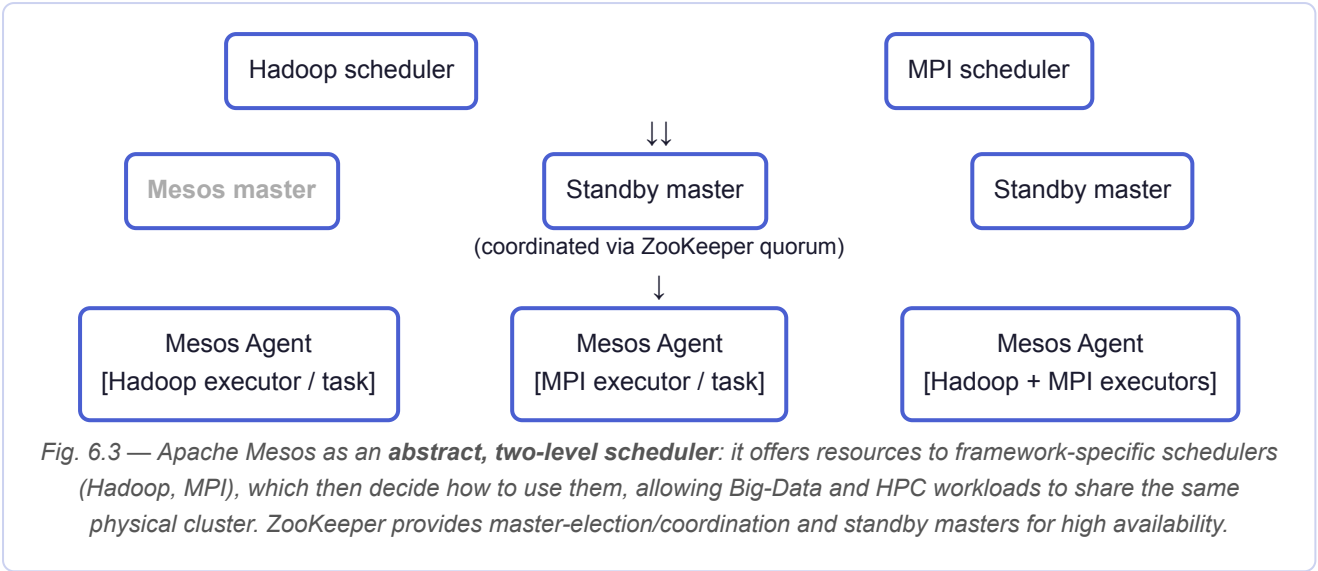
6.4 Software Stacks: Big Data vs. HPC ("Scale-out" vs "Scale-up" software ecosystems)

Layer	Data Analytics Ecosystem	Computational Science (HPC) Ecosystem
Application	Mahout, R, applications	Applications and community codes
Middleware / Management	Hive, Pig, Sqoop, Flume, MapReduce, Storm, HBase (key-value), HDFS; ZooKeeper (coordination); Avro (serialization); Cloud services (e.g., AWS)	FORTRAN/C/C++/IDEs, domain-specific libraries, MPI/OpenMP+accelerator tools, numerical libraries, performance & debugging tools (e.g., PAPI), Lustre parallel file system, batch schedulers (e.g., Slurm), system monitoring tools; Scientific visualisation
System software	Virtual machines and cloud services; containers (Kubernetes, Docker); Linux OS variant	Containers (Singularity, Shifter); Linux OS variant
Cluster hardware	Ethernet switches, local node storage, commodity x86 racks	InfiniBand + Ethernet, SAN + local node storage, x86 racks + GPUs/accelerators, in-situ processing

TAKE-AWAY Both ecosystems solve the same underlying scale-out problem but evolved independently with different priorities: Big Data optimises for *elastic, heterogeneous, cost-effective* commodity infrastructure and fault tolerance; HPC optimises for *tightly-coupled, low-latency, high-bandwidth* interconnects (InfiniBand) and numerically intensive workloads.

6.5 Scheduler Evolution: Convergence in Practice

Motivation: Big Data frameworks like Hadoop are not "just" data-intensive applications — they can also be seen as more general, *service-oriented* frameworks. A direct comparison to classic MPI batch jobs is therefore misleading; instead, scheduling should be **abstracted out** of any specific framework to allow flexible hardware provisioning across both worlds.



Diverse workload types on today's HPC systems

- **HPDA services and applications** (High Performance Data Analytics) — often short, loosely-coupled
- **Capacity supercomputing jobs** (large simulations) — long-running, tightly coupled, wide (up to 100k processes)
- **Interactive prototyping** — short, low process count
- **Job chains** — mapping dependencies between workflow steps

Plot dimensions: **x-axis** = number of processes per job (width, 1 → 100k), **y-axis** = job duration (seconds → weeks). "Tightly coupled parallelism" (e.g., Typical Capability Supercomputing Jobs — long duration, huge width) occupies the opposite corner from "loosely coupled parallelism" (e.g., Job Arrays, HPDA Applications — short, narrow-to-medium width). HPDA Services sit at long duration but low width.

Fig. 6.4 — Job diversity chart (after ScienceDirect 2017). Illustrates why a single, one-size-fits-all scheduler struggles to serve both extremes efficiently.

Management architecture

A generic scheduling/management architecture consists of: **Job Lifecycle Management** (job queues, queue-management policies, job monitoring, job log files) feeding into a **Scheduling** core (resource-allocation policies, resource allocation decisions, job assignment, job dispatch) which drives the **Compute Cluster**, all overseen by a **Resource Management** layer (current resource states, resource monitoring & management, resource management policy) — with both User and Administrator interfaces.

This surfaces an inherent **tension between usability** ("easy to use" submission for end users) **and the management perspective** (enforcing fair-share policies, quotas, priorities, and overall cluster health).

Scheduler performance comparison

Comparative study of major state-of-the-art schedulers — **Slurm vs. Grid Engine vs. Mesos vs. Hadoop YARN** — measuring scheduling overhead $\Delta T = T_{\text{total}} - T_{\text{job}}$ as a function of tasks-per-processor n :

Scheduler	Fitted overhead scaling
-----------	-------------------------

Slurm	$\Delta T \approx 2.2 \cdot n^{1.3}$
Grid Engine	$\Delta T \approx 2.8 \cdot n^{1.3}$
Mesos	$\Delta T \approx 3.4 \cdot n^{1.1}$
Hadoop YARN	$\Delta T \approx 33 \cdot n^1$

Modern HPC-native schedulers (Slurm, Grid Engine, Mesos) are up to a factor of ~10 faster than Hadoop YARN in terms of task-scheduling overhead. This matters most for workloads with **many small jobs**, or **long process chains in iterative analysis**, where per-task scheduling overhead accumulates.

EXAM ANGLE Be able to explain *why* scheduling overhead matters more for iterative/many-small-task workloads than for a few, large, long-running jobs: fixed per-task overhead is amortized over job runtime for big jobs, but dominates for many small/short tasks.

6.6 HPDA — Using HPC Idle Resources for Big Data Analytics

Core research question: **"Why use just one scheduler?"**

- Highly parallel HPC workloads require large parts of the system to be freed up (drained) before they can be scheduled (to get a large contiguous allocation).
- In principle, the resulting temporary "holes" in the schedule can be **filled with small, not time-dependent analytics jobs** — i.e., opportunistic co-location of Big Data jobs in HPC idle cycles.

Reference: M. Mercier, D. Glesser, Y. Georgiou, O. Richard, *"Big data and HPC collocation: Using HPC idle resources for Big Data analytics,"* IEEE Big Data 2017. Empirically shows that mixing HPC and Big Data workloads on the same system substantially increases overall core utilisation over time compared to running either workload type alone, at the cost of added scheduling complexity.

RESEARCH ANGLE This is a live, ongoing research area (not a solved problem): opportunistic co-scheduling requires careful interference management (I/O contention, network contention, memory bandwidth contention) so that "filler" Big Data jobs do not degrade the performance of the primary, tightly-coupled HPC simulation jobs they are packed around.

7. Master Comparison Cheat-Sheets

7.1 Framework Generations at a Glance

Generation	Representative system(s)	Core idea	Weakness addressed by next generation
1st — batch, disk-based	Hadoop MapReduce (+HDFS)	Scale-out, fault-tolerant batch processing of huge datasets on commodity clusters	Poor for iterative algorithms; heavy disk I/O between stages; only "map+reduce" primitives
2nd — ecosystem of specialised tools	Hive, Pig, Giraph, Oozie, HBase (on Hadoop 1.0)	Higher-level languages/paradigms (SQL, scripting, graph, workflow) reusing shared HDFS/MapReduce substrate	Still bound to MapReduce's disk-heavy execution model underneath
3rd — multi-purpose, in-memory	Apache Spark, Apache Flink	In-memory (RDD/dataflow) engines with DAG execution, unifying batch, SQL, streaming, graph, and ML in one stack	Spark's streaming is micro-batch (higher latency); true low-latency streaming still needed (Flink/Storm/Samza)

7.2 Stream Processing Engines

System	Streaming model	Latency	Throughput	API level	State handling
Storm	True streaming	Low	Lower	Low-level (Bolts/Spouts) + Trident	Limited
Spark Streaming	Micro-batching	Higher	High	Functional (DStreams)	Batch-runtime restricted
Samza	True streaming (on Kafka)	Low	High	Low-level	State is first-class citizen
Flink	True streaming, unifies batch as bounded stream	Low, tunable	High	Rich functional API	Exactly-once, checkpointed small state

7.3 HPC vs. Big Data — Summary Table

Dimension	HPC	Big Data
Typical job	Tightly-coupled simulation (MPI)	Loosely-coupled, data-parallel analytics
Scheduler	Slurm, Grid Engine, Moab, LoadLeveler	Hadoop Job/Task Tracker, YARN, Mesos
Interconnect	InfiniBand, low-latency fabrics	Ethernet, commodity networking
Storage	Lustre (parallel file system), SAN	HDFS (block-replicated, commodity disks)
Failure model	Rare, checkpoint/restart	Norm, transparently masked by framework

Programming	C/C++/Fortran + MPI/OpenMP	Java/Scala/Python + MapReduce/Spark APIs
Convergence point	Abstract schedulers (Mesos), HPDA co-location of idle-cycle Big-Data jobs, unified software stacks under containers (Singularity/Docker/Kubernetes)	

8. Exam & Research Q&A Bank

Questions are grouped by topic and tagged: **CONCEPT** core definitional understanding, **EXAM-STYLE** typical written-exam phrasing, **RESEARCH** open-ended / paper-adjacent reasoning.

8.1 Foundations & Scale-Out

1 Why does Big Data infrastructure prefer the scale-out paradigm over scale-up, given that a single powerful server is architecturally simpler? EXAM-STYLE

Cost of raw compute/storage/IO capacity scales roughly linearly with commodity hardware but super-linearly for high-end single machines (better CPUs, more memory channels, faster buses/interconnects get disproportionately expensive). Scale-out also naturally provides redundancy: since failure is assumed to be constant at scale, a system built from many independent, replaceable nodes can mask individual failures, whereas a scale-up system represents a single point of failure. The trade-off is added software complexity: the application/framework (not the hardware) must handle partitioning, coordination, and fault recovery.

2 Explain the four "V"s of Big Data and how each one influences pipeline design decisions. CONCEPT

Volume forces distributed storage/partitioning (HDFS blocks). **Velocity** forces stream/near-real-time processing choices (Flink/Storm rather than nightly batch). **Variety** forces schema-flexible storage/integration steps (the "Extraction/Cleaning/Annotation" and "Integration/Aggregation" pipeline stages exist largely because of variety). **Veracity** forces validation/cleaning steps and probabilistic/robust analysis methods, since noisy or unreliable data undermines downstream conclusions. Privacy and human interaction are cross-cutting constraints layered on top of all four.

8.2 Hadoop & MapReduce

3 List and explain the five core assumptions/goals behind the HDFS design. Why does each one make HDFS unsuitable as a drop-in replacement for a transactional database? EXAM-STYLE

(1) Hardware failure is the norm — HDFS assumes constant node/disk failure and replicates blocks automatically; a DB typically assumes rare failure and relies on RAID/backup. (2) Streaming data access — optimized for high sequential throughput, not random low-latency reads/writes required by OLTP. (3) Large data sets — tuned for large block sizes (historically 64–128MB) and millions of very large files, inefficient for many small records/rows. (4) Moving computation to data — the execution model (MapReduce) is designed around data-local scheduling, unlike a DB's client-issues-query model. (5) Portability — runs on heterogeneous commodity hardware, trading some performance for flexibility, unlike a DB tuned to specific hardware/storage engines. Together these make HDFS excellent for large-scale batch analytics but poor for low-latency transactional workloads (no ACID transactions, no efficient random updates).

4 In the MapReduce word-count example, which stage requires network-wide synchronization, and why is this typically the performance bottleneck? CONCEPT

The **shuffle** stage. Splitting and Mapping are "pure" parallel (each split/map task works purely on its local partition, no cross-node coordination). Shuffling must redistribute all intermediate key-value pairs across the network so that every value for a given key ends up at the same reducer — this requires sorting, network transfer, and often spilling to disk, making it I/O- and network-bound, unlike the embarrassingly-parallel map/reduce compute stages.

5 Geoffrey Fox identifies six MapReduce-derived computation patterns. Which two patterns most directly motivated the creation of Apache Spark, and why? RESEARCH

Pattern 3 (Iterative MapReduce / Map-Collective) and Pattern 6 (Shared-memory Map-Communicates). Classic MapReduce (Pattern 2) forces a fresh read from and write to disk (typically HDFS) between every map-reduce round; for algorithms that iterate many times over the same dataset (e.g., k-means, PageRank, gradient descent), this disk round-trip cost is repeated needlessly on every iteration. Spark's RDDs directly address this by keeping data cached in memory across iterations (Pattern 6's "shared memory" idea) while still supporting the iterative control flow of Pattern 3 — this is precisely why Spark's own benchmark shows growing speedup over Hadoop as the number of iterations increases.

6 Why is MapReduce classified as an SIMD-style problem when contrasted with MPI/HPC workloads? EXAM-STYLE

In MapReduce, the exact same map function (a single "instruction stream") is applied uniformly and independently to many different pieces of data (multiple data streams) — matching the Single-Instruction-Multiple-Data pattern. Classic MPI/HPC programs, by contrast, are typically MIMD: independent processes can run different code paths and communicate arbitrarily (point-to-point, collective operations), giving far more flexibility but also far more complexity than MapReduce's constrained model.

8.3 In-Memory Processing

7 What was the historical business driver for in-memory processing, and what does "coffee-break analytics" refer to? CONCEPT

Business Intelligence (BI). Classical disk/relational-DB-based analytic reports were slow enough that a user would submit a query, then literally go get a coffee while waiting for results ("coffee-break analytics"). Falling RAM prices made it economically feasible to keep working datasets resident in memory at scale, removing the IO-wait bottleneck and enabling near-real-time analysis.

8 Caching, replication, and prefetching are all hardware-level techniques to hide IO latency. Give one concrete limitation for each, and explain why none of them alone is sufficient for general Big Data analytics workloads. EXAM-STYLE

Caching only helps if the "working set" is small enough to fit in the cache and is accessed repeatedly — a full scan over a petabyte-scale dataset does not benefit. Prefetching only helps if access patterns are predictable; ad-hoc analytical queries often have irregular access patterns. Replication is expensive (extra storage + write overhead) and bounded by receiving-side machine capacity. Because their effectiveness is workload-dependent, none is a general solution; this is exactly why full in-memory *processing frameworks* (like Spark, which explicitly caches computed RDDs based on the application's own knowledge of reuse) outperform relying on generic hardware-level tricks alone.

8.4 Apache Spark

9 Define an RDD and explain how it achieves fault tolerance without needing full data replication. CONCEPT

An RDD (Resilient Distributed Dataset) is an immutable, partitioned collection of records distributed across a cluster. Fault tolerance comes from the **lineage graph**: Spark records the sequence of deterministic transformations used to derive each RDD from its parent(s)/source data. If a partition is lost (e.g., a worker node fails), Spark can *recompute* just that partition by re-applying the recorded transformations to the still-available parent data/source, rather than needing to keep redundant physical replicas of every intermediate result (as HDFS does for raw storage).

10 Distinguish transformations from actions in Spark, and explain the practical consequence of transformations being lazy. EXAM-STYLE

Transformations (`map` , `filter` , ...) produce a new RDD description but do not execute anything immediately — they are lazily queued. Actions (`collect` , `reduce` , `saveAsTextFile` , ...) trigger actual execution of the entire chain of pending transformations and return/persist a concrete result. Because execution is deferred, Spark can see the *whole* DAG of operations before running anything, allowing it to optimise the physical execution plan (e.g., pipeline several narrow transformations into a single pass, avoid materializing intermediate results that are never separately needed, and schedule tasks to maximise data locality).

11 Why did Spark introduce DataFrames and then Datasets on top of the original RDD API? What specific engineering techniques (since v1.4) made this possible? RESEARCH

RDDs operate on arbitrary, opaque JVM objects — Spark's optimizer has no visibility into their internal structure, so it cannot apply relational-style query optimisation. DataFrames introduce a known schema (rows of typed columns), enabling Spark's Catalyst optimizer to build and rewrite logical query plans much like a traditional database engine — but at the cost of losing compile-time type safety (errors surface only at runtime). Datasets recover type safety by using typed JVM objects *externally* while representing data internally as rows, thus getting DataFrame-style optimisation "for free" while still catching type errors at compile time — the "best of both worlds." This was enabled by the **Tungsten execution engine**: explicit, application-aware (schema-aware) binary memory management that bypasses JVM object/garbage-collection overhead; cache-aware algorithms/data structures that exploit the CPU cache hierarchy; and runtime code generation that compiles expression trees into specialized bytecode rather than interpreting them generically.

12 Why does each Spark application get its own private driver and executors, and what is a downside of this isolation model? RESEARCH

Per-application isolation ensures that one application's failures, resource usage, or bugs cannot directly corrupt or starve another concurrently-running application on the same cluster — each has independently managed executors and its own scheduling within the cluster manager's overall resource allocation. The downside is that this makes it harder to *share* cached, in-memory data between separate applications: caching benefits are scoped per-application, so if two different jobs need the same hot dataset, each must independently load/cache it (unless an external shared store — e.g., Alluxio/Tachyon — is used to bridge that gap).

8.5 Streaming: Spark vs. Flink

13 Explain the fundamental architectural difference between Spark Streaming and Apache Flink's streaming model, and why this leads to different latency characteristics. EXAM-STYLE

Spark Streaming implements streaming as a sequence of small **micro-batches** (DStreams): incoming data is buffered over a short interval, then processed as a normal (small) Spark batch job, and results emitted at the batch boundary. Flink implements **native/true streaming**: each event is processed by the runtime as it arrives, with results able to flow continuously between operators without waiting for an artificial batch boundary. Because Spark's minimum latency is bounded below by its micro-batch interval (you cannot get a result faster than one batch period), while Flink's latency is determined per-event by the actual processing pipeline, Flink can offer lower and more finely tunable latency, at a comparable throughput, for the same workload — as confirmed empirically by Karimov et al.'s ICDE 2018 benchmark, which found a slight latency offset for Spark relative to Flink at equivalent throughput and cluster scale.

14 Why does Flink treat batch processing as a "special case" of stream processing rather than the reverse? What does this unification buy Flink architecturally? RESEARCH

A batch is simply a stream that happens to be *bounded* (has a known end), whereas a stream in the general case is *unbounded*. Modeling batch as bounded-stream lets Flink use one single execution engine and API surface for both ETL/graph/ML batch workloads and low-latency windowed streaming workloads (as shown in the "Flink in the Analytics Ecosystem" diagram, where both event logs and historic data flow through the same Flink core). Spark instead does the reverse — it builds streaming (DStreams) as a layer of repeated small batches on top of a fundamentally batch-oriented core — which is architecturally simpler to build initially but structurally limits how low latency can go.

15 What does "exactly-once processing guarantee with small state" mean in the context of Flink, and why is this a hard problem in a distributed streaming system? CONCEPT

"Exactly-once" means each input event affects the final result state precisely once — not zero times (data loss on failure) and not more than once (duplicate processing after a restart/failure). This is hard because streaming state (e.g., a running aggregate/window) is continuously mutated across many distributed operators; if a node fails mid-stream, the system must be able to restore state and replay only the exact right set of unprocessed events, without either losing updates or double-applying them. Flink achieves this via distributed, consistent checkpointing of operator state (e.g., using the Chandy–Lamport-style snapshot algorithm), keeping the checkpointed state deliberately small/efficient so checkpointing does not itself become a throughput bottleneck.

8.6 HPC and Big Data Convergence

16 Explain why comparing a Hadoop job directly to an MPI job is described in the lecture as "misleading." EXAM-STYLE

MPI jobs are typically tightly-coupled, long-running, tightly-synchronized parallel simulations that need a large block of dedicated, homogeneously-allocated resources for their full duration. Hadoop-class Big Data frameworks are better understood as *service-oriented* frameworks (long-lived clusters continuously accepting and scheduling many independent, loosely-coupled analytics tasks), not as single monolithic parallel jobs. Comparing "one Hadoop job" to "one MPI job" therefore compares the wrong units — the meaningful comparison is at the level of the underlying scheduling/resource-management abstraction (which is why frameworks like Mesos try to abstract scheduling out of any single framework, exposing raw resources to both Hadoop and MPI schedulers).

17 Why do modern schedulers like Slurm and Mesos have measured task-scheduling overhead roughly an order of magnitude lower than Hadoop YARN, and in which workloads does this matter most? RESEARCH

The benchmarked overhead $\Delta T = T_{\text{total}} - T_{\text{job}}$ scales as $\Delta T \approx c \cdot n^k$ in tasks-per-processor n , with fitted constants roughly 2.2–3.4 for Slurm/Grid Engine/Mesos versus about 33 for Hadoop YARN (with a similar exponent). Even though the underlying causes are implementation-specific (YARN's resource-negotiation protocol involves more overhead per task dispatch/heartbeat than HPC-native schedulers optimized for rapid task turnover), the practical consequence is the same: this overhead is a roughly fixed cost paid *per task*. For a small number of large, long-running jobs, this fixed per-task cost is negligible relative to total runtime. But for workloads composed of **many small tasks** or **long chains of dependent tasks** in iterative analysis pipelines, the accumulated overhead can become a dominant fraction of total wall-clock time — exactly the HPDA/iterative-analytics scenario the lecture highlights.

18 Describe the core idea behind using HPC idle resources for Big Data analytics (HPDA), and identify one practical risk of this approach. RESEARCH

Highly parallel, tightly-coupled HPC jobs often require the scheduler to drain/free large contiguous blocks of the cluster before they can start, temporarily leaving "holes" of otherwise-idle resources. Mercier et al. (IEEE Big Data 2017) propose deliberately filling these holes with small, not-time-critical Big Data analytics jobs, improving overall cluster utilisation without displacing the primary HPC workload. The practical risk is **interference**: even though the "filler" jobs are scheduled into ostensibly idle compute slots, they can still compete for shared resources — network bandwidth, parallel filesystem I/O, memory bandwidth, or cache — with neighbouring, still-running HPC jobs, potentially causing subtle performance degradation ("noisy neighbour" effects) in the primary simulation workloads that the co-location was never supposed to affect.

19 Contrast the software stacks of the "Data Analytics Ecosystem" and the "Computational Science Ecosystem" at the cluster-hardware level, and explain the underlying design trade-off. EXAM-STYLE

The Data Analytics ecosystem typically runs on Ethernet-switched, commodity x86 racks with local node storage — optimizing for low cost per node and horizontal elasticity, tolerating higher network latency because workloads (MapReduce/Spark-style) are designed around data-local, loosely-coupled tasks with limited cross-node synchronization. The Computational Science (HPC) ecosystem uses InfiniBand plus Ethernet, SAN plus local storage, and x86 racks augmented with GPUs/accelerators (plus in-situ processing) — optimizing for very low-latency, high-bandwidth interconnects, because tightly-coupled MPI simulations require frequent, fine-grained synchronization/communication across many nodes where network latency directly limits scalability (Amdahl's-law-style bottlenecks). The trade-off is cost vs. communication performance: HPC hardware is more expensive per node but essential for strongly-coupled numerical simulation; commodity Big Data hardware is cheaper and scales well specifically because its workloads were designed to minimize cross-node communication in the first place.

20 Using the "Theories–Deduction–Predictions–Induction–Abduction–Observations" cycle, explain where HPC and HDA (Big-Data analytics) each contribute, and why real scientific workflows increasingly need both. RESEARCH

HPC contributes on the **deductive** side: given a theory formulated as a mathematical model, HPC executes that model (large-scale simulation) to generate predictions. HDA (High-end/High-Performance Data Analytics) contributes on the **inductive and abductive** side: assimilating predictions together with real observational data to evaluate the hypothesis (induction), and discovering patterns/anomalies in observational data to generate new explanatory hypotheses in the first place (abduction). Modern scientific discovery increasingly needs both in the same workflow — e.g., a climate model (HPC simulation) whose predictions must be continuously validated against huge, heterogeneous sensor/observation datasets (HDA), with anomalies detected in that data potentially feeding back to refine the model's theoretical assumptions. This is precisely the motivation behind convergence infrastructure (shared schedulers, shared software stacks) discussed in this lecture — neither pure HPC nor pure Big Data tooling alone is sufficient for such iterative, mixed workflows.

8.7 Cross-Cutting Synthesis Questions

21 Trace the evolution from Google's original MapReduce paper (2004) to Apache Flink, identifying the single biggest limitation solved at each step. RESEARCH

MapReduce (2004): introduced fault-tolerant, scale-out batch processing on commodity clusters — solving the "how do I process petabytes reliably on cheap hardware" problem, but at the cost of heavy disk I/O between every map-reduce stage. **Hadoop ecosystem (Hive/Pig/Giraph/etc., ~2008-2010):** solved the "MapReduce's map/reduce primitives are too low-level for many use cases" problem by adding higher-level languages/paradigms on the same substrate — but all still ultimately compiled down to disk-heavy MapReduce jobs. **Apache Spark (2010-2014):** solved the "iterative and interactive workloads are punishingly slow on MapReduce" problem via in-memory RDDs with lineage-based fault tolerance and a DAG scheduler — but its streaming support (added via DStreams) remained fundamentally micro-batch, so low-latency streaming was still imperfect. **Apache Flink:** solved the "streaming latency is bounded by batch-interval" problem via a native streaming runtime treating batch as bounded-stream, with exactly-once state guarantees.

22 Why might an exam question frame Apache Mesos as "the Spark of scheduling"? Justify or refute this analogy. RESEARCH

The analogy is reasonably apt at a structural level: just as Spark provides one common in-memory execution engine (RDD/DAG) beneath multiple higher-level libraries (SQL, Streaming, GraphX, MLlib) that previously would have been separate, framework-specific systems, Mesos provides one common resource-abstraction layer beneath multiple higher-level, framework-specific schedulers (a Hadoop scheduler, an MPI scheduler, etc.) that would otherwise each need to manage cluster hardware independently. Both are examples of "unify the substrate, let specialised logic sit on top." The analogy should not be taken too literally, however: Spark unifies *data processing semantics* (a shared, optimizable data abstraction), while Mesos only unifies *resource offers* (raw CPU/memory/disk slots) — it does not attempt to unify or optimize the actual computation happening inside each framework the way Spark's Catalyst/Tungsten stack does.

23 A start-up wants to build a real-time fraud-detection system processing a continuous stream of credit-card transactions with sub-100ms latency requirements, plus a nightly batch job re-training its fraud model on the full historical dataset. Which technologies from this lecture would you recommend, and why? EXAM-STYLE

For the sub-100ms real-time path: **Apache Flink**, because its native/true streaming model (as opposed to Spark Streaming's micro-batching) is specifically suited to tight, tunable latency requirements, and its exactly-once state guarantees are important for a financial use case where double-counting or dropping a transaction has real consequences. For the nightly batch re-training job: either classic **Hadoop MapReduce** or, more likely today, **Apache Spark** (with MLlib), since batch retraining over the full historical dataset is not latency-critical but does benefit from Spark's in-memory, iterative-friendly engine (training algorithms like gradient descent are inherently iterative — exactly the case where Spark's in-memory caching massively outperforms disk-bound MapReduce). Underlying storage would be **HDFS** (or a cloud-native equivalent) given the "large data sets, streaming/high-throughput access" assumptions match a historical transaction archive well, with Kafka as the ingestion layer feeding both the Flink real-time path and periodic dumps into HDFS for the batch path.

24 Explain, in one coherent argument, why "moving computation to data" (an HDFS design goal) and "keeping data resident in memory" (Spark's core idea) are really two instances of the same underlying principle. RESEARCH

Both techniques minimise the same fundamental cost: **moving large volumes of data across a slow channel** (network or disk I/O) relative to moving a comparatively tiny amount of code or keeping data where it already sits. HDFS's "move computation to data" avoids shipping large data blocks over the network by instead shipping the (small) map function to the node that already holds the relevant HDFS block. Spark's in-memory RDD caching avoids the analogous cost of repeatedly re-reading data from disk on every iteration by keeping the working set resident in RAM once it has been computed/loaded — i.e., it avoids "moving data" (from disk to memory) more than once. Both are manifestations of the general Big-Data engineering principle: *data movement, not computation, is usually the true bottleneck at scale*, so systems are designed to minimise redundant data movement (across network, or across the memory hierarchy) wherever possible.

Compiled study guide based on "Digitization and Data Analytics" (Distributed Computing II, Lecture 11), TU Dresden CIDS/ZIH. All figures redrawn/paraphrased from the original slide content for study purposes.